Configuring parameters and parameter data types

Command objects use parameters to pass values to SQL statements or stored procedures, providing type checking and validation. Unlike command text, parameter input is treated as a literal value, not as executable code. This helps guard against "SQL injection" attacks, in which an attacker inserts a command that compromises security on the server into an SQL statement.

Parameterized commands can also improve query execution performance, because they help the database server accurately match the incoming command with a proper cached query plan. For more information, see Execution Plan Caching and Reuse and Parameters and Execution Plan Reuse. In addition to the security and performance benefits, parameterized commands provide a convenient method for organizing values passed to a data source.

A DbParameter object can be created by using its constructor, or by adding it to the DbParameterCollection by calling the `Add` method of the DbParameterCollection collection. The `Add` method will take as input either constructor arguments or an existing parameter object, depending on the data provider.

Supplying the ParameterDirection property

When adding parameters, you must supply a ParameterDirection property for parameters other than input parameters. The following table shows the `ParameterDirection` values that you can use with the ParameterDirection enumeration.

Expand table

Member name	Description
Input	The parameter is an input parameter. This is the default.
InputOutput	The parameter can perform both input and output.
Output	The parameter is an output parameter.
ReturnValue	The parameter represents a return value from an operation such as a stored procedure, built-in function, or user-defined function.

Working with parameter placeholders

The syntax for parameter placeholders depends on the data source. The .NET Framework data providers handle naming and specifying parameters and parameter placeholders differently. This syntax is customized to a specific data source, as described in the following table.

Expand table

Data provider	Parameter naming syntax
Microsoft.Data.SqlClient	Uses named parameters in the format `@parametername`.
System.Data.OleDb	Uses positional parameter markers indicated by a question mark (`?`).
System.Data.Odbc	Uses positional parameter markers indicated by a question mark (`?`).
System.Data.OracleClient	Uses named parameters in the format `:parmname` (or *parmname*).

Specifying parameter data types

The data type of a parameter is specific to the .NET Framework data provider. Specifying the type converts the value of the `Parameter` to the .NET Framework data provider type before passing the value to the data source. You may also specify the type of a `Parameter` in a generic manner by setting the `DbType` property of the `Parameter` object to a particular DbType.

The .NET Framework data provider type of a `Parameter` object is inferred from the .NET Framework type of the `Value` of the `Parameter` object, or from the `DbType` of the `Parameter` object. The following table shows the inferred `Parameter` type based on the object passed as the `Parameter` value or the specified `DbType`.

Expand table

.NET Framework type	DbType	SqlDbType	OleDbType	OdbcType
Boolean	Boolean	Bit	Boolean	Bit
Byte	Byte	TinyInt	UnsignedTinyInt	TinyInt
byte[]	Binary	VarBinary. This implicit conversion will fail if the byte array is larger than the maximum size of a VarBinary, which is 8000 bytes.For byte arrays larger than 8000 bytes, explicitly set the SqlDbType.	VarBinary	Binary
Char		Inferring a SqlDbType from char is not supported.	Char	Char
DateTime	DateTime	DateTime	DBTimeStamp	DateTime
DateTimeOffset	DateTimeOffset	DateTimeOffset in SQL Server 2008. Inferring a SqlDbType from DateTimeOffset is not supported in versions of SQL Server earlier than SQL Server 2008.		
Decimal	Decimal	Decimal	Decimal	Numeric
Double	Double	Float	Double	Double
Single	Single	Real	Single	Real
Guid	Guid	UniqueIdentifier	Guid	UniqueIdentifier

.NET Framework type	DbType	SqlDbType	OleDbType	OdbcType
Int16	Int16	SmallInt	SmallInt	SmallInt
Int32	Int32	Int	Int	Int
Int64	Int64	BigInt	BigInt	BigInt
Object	Object	Variant	Variant	Inferring an OdbcType from Object is not supported.
String	String	NVarChar. This implicit conversion will fail if the string is larger than the maximum size of an NVarChar, which is 4000 characters. For strings larger than 4000 characters, explicitly set the SqlDbType.	VarWChar	NVarChar
TimeSpan	Time	Time in SQL Server 2008. Inferring a SqlDbType from TimeSpan is not supported in versions of SQL Server earlier than SQL Server 2008.	DBTime	Time
UInt16	UInt16	Inferring a SqlDbType from UInt16 is not supported.	UnsignedSmallInt	Int
UInt32	UInt32	Inferring a SqlDbType from UInt32 is not supported.	UnsignedInt	BigInt
UInt64	UInt64	Inferring a SqlDbType from UInt64 is not supported.	UnsignedBigInt	Numeric
	AnsiString	VarChar	VarChar	VarChar
	AnsiStringFixedLength	Char	Char	Char
	Currency	Money	Currency	Inferring an `OdbcType` from `Currency` is not supported.
	Date	Date in SQL Server 2008. Inferring a SqlDbType from Date is not supported in versions of SQL Server earlier than SQL Server 2008.	DBDate	Date
	SByte	Inferring a SqlDbType from SByte is not supported.	TinyInt	Inferring an `OdbcType` from SByte is not supported.
	StringFixedLength	NChar	WChar	NChar
	Time	Time in SQL Server 2008. Inferring a SqlDbType from Time is not supported in versions of SQL Server earlier than SQL Server 2008.	DBTime	Time

.NET Framework type	DbType	SqlDbType	OleDbType	OdbcType
	VarNumeric	Inferring a SqlDbType from VarNumeric is not supported.	VarNumeric	Inferring an `OdbcType` from VarNumeric is n supported.
user-defined type (an object with SqlUserDefinedAggregateAttribute	Object or String, depending the provider (SqlClient always returns an Object, Odbc always returns a String, and the OleDb managed data provider can see either	SqlDbType.Udt if SqlUserDefinedTypeAttribute is present, otherwise Variant	OleDbType.VarWChar (if value is null) otherwise OleDbType.Variant.	OdbcType.NVar(

> **Note**
>
> Conversions from decimal to other types are narrowing conversions that round the decimal value to the nearest integer value toward zero. If the result of the conversion is not representable in the destination type, an **OverflowException** is thrown.

> **Note**
>
> When you send a null parameter value to the server, you must specify **DBNull**, not `null` (`Nothing` in Visual Basic). The null value in the system is an empty object that has no value. **DBNull** is used to represent null values. For more information about database nulls, see Handling Null Values.

Deriving parameter information

Parameters can also be derived from a stored procedure using the `DbCommandBuilder` class. Both the `SqlCommandBuilder` and `OleDbCommandBuilder` classes provide a static method, `DeriveParameters`, which automatically populates the parameters collection of a command object that uses parameter information from a stored procedure. Note that `DeriveParameters` overwrites any existing parameter information for the command.

> **Note**
>
> Deriving parameter information incurs a performance penalty because it requires an additional round trip to the data source to retrieve the information. If parameter information is known at design time, you can improve the performance of your application by setting the parameters explicitly.

For more information, see Generating Commands with CommandBuilders.

Using parameters with a SqlCommand and a stored procedure

Stored procedures offer many advantages in data-driven applications. By using stored procedures, database operations can be encapsulated in a single command, optimized for best performance, and enhanced with additional security. Although a stored procedure can be called by passing the stored procedure name followed by parameter arguments as an SQL statement, by using the Parameters collection of the ADO.NET DbCommand object enables you to more explicitly define stored procedure parameters, and to access output parameters and return values.

> **Note**
>
> Parameterized statements are executed on the server by using `sp_executesql`, which allows for query plan reuse. Local cursors or variables in the `sp_executesql` batch are not visible to the batch that calls `sp_executesql`. Changes in database context last only to the end of the `sp_executesql` statement. For more information, see **sp_executesql (Transact-SQL)**.

When using parameters with a SqlCommand to execute a SQL Server stored procedure, the names of the parameters added to the Parameters collection must match the names of the parameter markers in the stored procedure. Microsoft.Data.SqlClient does not support the question mark (?) placeholder for passing parameters to an SQL statement or a stored procedure. It treats parameters in the stored procedure as named parameters and searches for matching parameter markers. For example, the `CustOrderHist` stored procedure is defined by using a parameter named `@CustomerID`. When your code executes the stored procedure, it must also use a parameter named `@CustomerID`.

SQL

```
CREATE PROCEDURE dbo.CustOrderHist @CustomerID varchar(5)
```

Example

This example demonstrates how to call a SQL Server stored procedure in the `Northwind` sample database. The name of the stored procedure is `dbo.SalesByCategory` and it has an input parameter named `@CategoryName` with a data type of `nvarchar(15)`. The code creates a new SqlConnection inside a using block so that the connection is disposed when the procedure ends. The SqlCommand and SqlParameter objects are created, and their properties set. A SqlDataReader executes the `SqlCommand` and returns the result set from the stored procedure, displaying the output in the console window.

> **Note**
>
> Instead of creating `SqlCommand` and `SqlParameter` objects and then setting properties in separate statements, you can instead elect to use one of the overloaded constructors to set multiple properties in a single statement.

C#

```
static void GetSalesByCategory(string connectionString,
    string categoryName)
```

```
{
    using (SqlConnection connection = new(connectionString))
    {
        // Create the command and set its properties.
        SqlCommand command = new()
        {
            Connection = connection,
            CommandText = "SalesByCategory",
            CommandType = CommandType.StoredProcedure
        };

        // Add the input parameter and set its properties.
        SqlParameter parameter = new()
        {
            ParameterName = "@CategoryName",
            SqlDbType = SqlDbType.NVarChar,
            Direction = ParameterDirection.Input,
            Value = categoryName
        };

        // Add the parameter to the Parameters collection.
        command.Parameters.Add(parameter);

        // Open the connection and execute the reader.
        connection.Open();
        using (SqlDataReader reader = command.ExecuteReader())
        {
            if (reader.HasRows)
            {
                while (reader.Read())
                {
                    Console.WriteLine($"{reader[0]}: {reader[1]:C}");
                }
            }
            else
            {
                Console.WriteLine("No rows found.");
            }
            reader.Close();
        }
    }
}
```

Using parameters with an OleDbCommand or OdbcCommand

When using parameters with an OleDbCommand or OdbcCommand, the order of the parameters added to the `Parameters` collection must match the order of the parameters defined in your stored procedure. The .NET Framework Data Provider for OLE DB and .NET Framework Data Provider for ODBC treat parameters in a stored procedure as placeholders and apply parameter values in order. In addition, return value parameters must be the first parameters added to the `Parameters` collection.

The .NET Framework Data Provider for OLE DB and .NET Framework Data Provider for ODBC do not support named parameters for passing parameters to an SQL statement or a stored procedure. In this case, you must use the question mark (?) placeholder, as in the following example.

SQL

```
SELECT * FROM Customers WHERE CustomerID = ?
```

As a result, the order in which `Parameter` objects are added to the `Parameters` collection must directly correspond to the position of the ? placeholder for the parameter.

OleDb Example

C#

```
OleDbCommand command = new OleDbCommand("SampleProc", connection);
command.CommandType = CommandType.StoredProcedure;

OleDbParameter parameter = command.Parameters.Add(
  "RETURN_VALUE", OleDbType.Integer);
parameter.Direction = ParameterDirection.ReturnValue;

parameter = command.Parameters.Add(
  "@InputParm", OleDbType.VarChar, 12);
parameter.Value = "Sample Value";

parameter = command.Parameters.Add(
  "@OutputParm", OleDbType.VarChar, 28);
parameter.Direction = ParameterDirection.Output;
```

Odbc Example

C#

```
OdbcCommand command = new OdbcCommand( _
  "{ ? = CALL SampleProc(?, ?) }", connection);
command.CommandType = CommandType.StoredProcedure;

OdbcParameter parameter = command.Parameters.Add( _
  "RETURN_VALUE", OdbcType.Int);
parameter.Direction = ParameterDirection.ReturnValue;

parameter = command.Parameters.Add( _
  "@InputParm", OdbcType.VarChar, 12);
parameter.Value = "Sample Value";

parameter = command.Parameters.Add( _
  "@OutputParm", OdbcType.VarChar, 28);
parameter.Direction = ParameterDirection.Output;
```

See also

- Commands and Parameters
- DataAdapter Parameters
- Data Type Mappings in ADO.NET
- ADO.NET Overview

Last updated on 19/10/2026